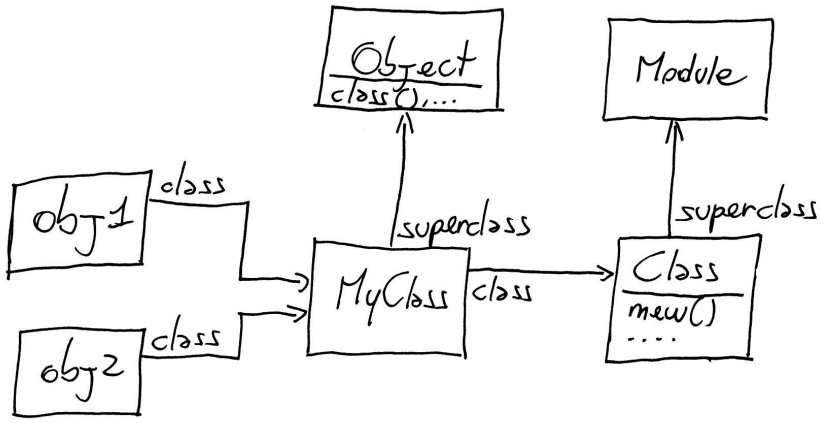


Figure 2—Classes are just objects.

"See?" Bill asks, pointing at the previous diagram. "Classes and regular objects live together happily."

There's one more interesting detail in the "Classes are objects" theme: like you do with any other object, you hold onto a class with a *reference*. A variable can reference a class just like any other object:

```
my_class = MyClass
```

`MyClass` and `my_class` are both references to the same instance of `Class`—the only difference being that `my_class` is a variable, while `MyClass` is a constant. To put this differently, just as classes are nothing but objects, class names are nothing but constants. So let's look more closely at constants.

Constants

Any reference that begins with an uppercase letter, including the names of classes and modules, is a *constant*. You might be surprised to learn that a Ruby constant is actually very similar to a variable—to the extent that you can change the value of a constant, although you will get a warning from the interpreter. (If you're in a destructive mood, you can even break Ruby beyond repair by changing the value of the `String` class name.)

If you can change the value of a constant, how is a constant different from a variable? The one important difference has to do with their scope. The scope of constants follows its own special rules, as you can see in the example that follows.